

Faculty of Computing

Sri Lanka Institute of Information Technology

2026 Year 2 Semester 1

IT2011 – Artificial Intelligence and Machine Learning

Final Group Report

Group ID: 2026-Y2-S1-MET-16

Company Bankruptcy Prediction

Group Members-

Name	IT Number	Preprocessing Contribution	ML Model
Sandeepana A. G. P.	IT25102100	Feature engineering	XGBoost
Raveenath K. A. Y.	IT25102101	Feature scaling	KNN
Rathnaweera R. P. S. D.	IT25102102	Remove irrelevant/constant columns, rename columns	Gradient Boosting
Adhikari A. A. P. P.	IT25102103	Missing values and duplicate rows	Naive Bayes
Parindya K. P.	IT25102104	Outlier removal using IQR	SVM
De Silva S. L. O. D.	IT25102105	Correlation heatmap and feature selection	AdaBoost

1. Introduction and Problem Statement

1.1 Background

A company's bankruptcy can lead to losses for investors, lenders, employees and suppliers. The detection of financially distressed companies, therefore, plays an important role in credit and investment analyses. Financial statements include ratios of profitability, leverage, liquidity and cash flow, and these can be used to predict bankruptcy. Traditional scoring systems such as Altman's Z-score have been developed for this reason. Machine-learning models, however, can learn patterns that link numerous financial ratios, including non-linear relationships.

1.2 Problem Statement

The task is to predict whether a company is bankrupt or not, based on its financial ratios. This is a supervised, binary classification problem where class 1 is bankrupt and class 0 is not bankrupt. The main difficulty is that the classes are imbalanced; bankrupt companies make up a small fraction of the dataset. As a result, a predictive model can exhibit high accuracy, even if it fails to detect many bankrupt companies.

1.3 Aim and Objectives

The aim is to build and benchmark six machine-learning classifiers for company bankruptcy prediction and determine the most appropriate one.

- Explore the dataset and understand the class imbalance.
- Apply the agreed preprocessing steps including cleaning, outlier handling, scaling, feature selection and feature engineering.
- Train SVM, XGBoost, KNN, Gradient Boosting, AdaBoost and Gaussian Naive Bayes on the same train/test data.
- Handle class imbalance and tuning using cross-validation.
- Benchmark the models using precision, recall, F1, ROC-AUC and PR-AUC, and discuss limitations and possible improvements.

1.4 End-to-End Workflow

The project followed one workflow from raw data to final model comparison (Figure 1). The raw data were processed in six steps and split once into a training set and a test set; all six models were then trained on the same split. Each model was tuned with cross-validation on the training data only, and the test set was used once for final benchmarking.

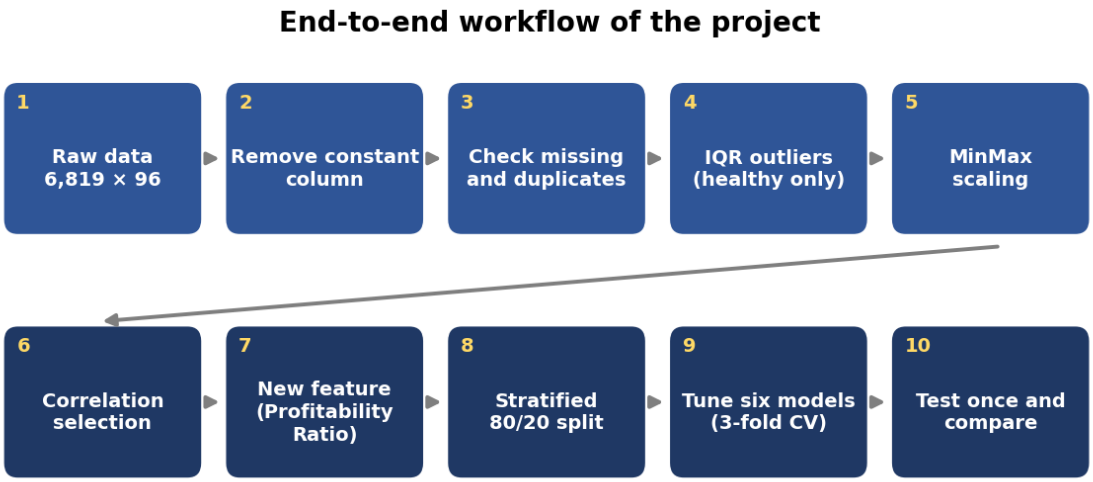


Figure 1. End-to-end workflow of the project, from the raw dataset to the final model comparison.

2. Dataset Description

The dataset is the Company Bankruptcy Prediction dataset based on annual financial ratios from the Taiwanese Economic Journal for 1999–2009. Each row is a company and the target column is Bankrupt?

Attribute	**Raw dataset**	**After preprocessing**
Rows	6,819	6,198
Columns	96 (95 features + target)	64 (63 features + target)
Target	0 = not bankrupt, 1 = bankrupt	Same
Class 0 / Class 1	6,599 / 220	5,978 / 220
Bankrupt share	3.23%	3.55%
Missing / duplicate rows	0 / 0	0 / 0
Problem type	Binary classification	Same

The features include profitability, leverage, solvency, liquidity, cash-flow, per-share and turnover ratios. Net Income Flag had the same value for every row and did not provide useful information for prediction.

Source. The data come from the UCI Machine Learning Repository (Taiwanese Bankruptcy Prediction dataset, https://archive.ics.uci.edu/dataset/572/taiwanese+bankruptcy+prediction) and were used in Liang et al. (2016). The data were collected from the Taiwanese Economic Journal for 1999–2009.

Feature groups. The table gives examples of the 95 original features. The groups are a simple classification made by the team to describe the data.

Group	Example features
Profitability	ROA(A), ROA(B), ROA(C), Operating Gross Margin, Net Income to Total Assets
Liquidity and solvency	Current Ratio, Quick Ratio, Debt ratio %, Cash/Total Assets
Leverage	Total debt/Total net worth, Liability to Equity, Degree of Financial Leverage (DFL)
Cash flow and per-share	Cash Flow to Total Assets, Persistent EPS in the Last Four Seasons, Revenue Per Share
Turnover (efficiency)	Total Asset Turnover, Inventory Turnover Rate, Accounts Receivable Turnover
Growth	Total Asset Growth Rate, Net Value Growth Rate, Operating Profit Growth Rate

Class balance. Figure 2 shows the class balance at different steps. In the raw data 6,599 companies are not bankrupt and 220 are bankrupt (3.23%). After preprocessing the numbers are 5,978 and 220 (3.55%), since only healthy companies were removed as outliers. The stratified split keeps the same share in the training set (4,782 healthy and 176 bankrupt) and in the test sets (1,196 healthy and 44 bankrupt). In the training set there are roughly 27 healthy companies for each bankrupt one, so the models can easily favour the healthy class.

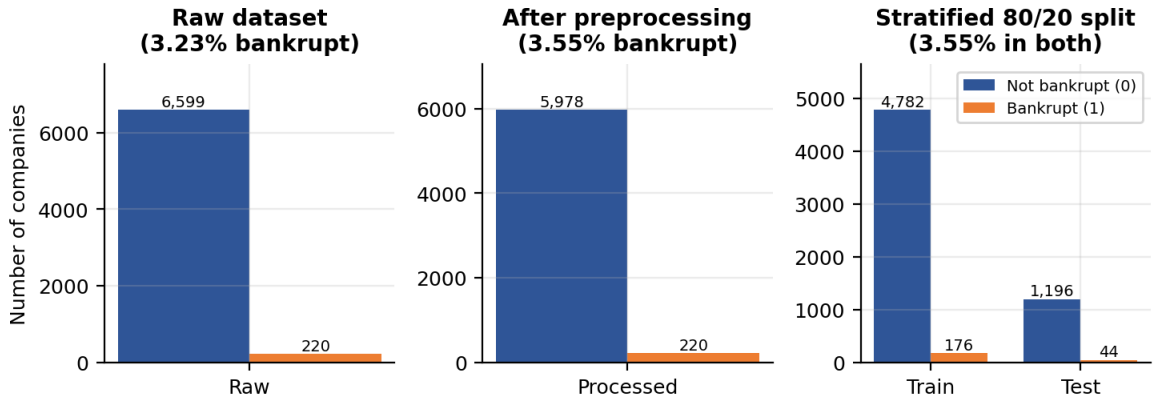


Figure 2. Class balance in the raw data, after preprocessing, and in the training and test sets.

3. Preprocessing and Exploratory Data Analysis

The group used a seven-step preprocessing pipeline. Each member was responsible for one step, and the final processed data were used by all six models to ensure fairness in benchmarking.

Step	What was done	Reason / effect
1. Constant columns and renaming	Cleaned headers and removed Net Income Flag.	A constant column cannot help prediction. 96 → 95 columns.
2. Missing values and duplicates	Checked nulls and duplicates.	The dataset had 0 missing values and 0 duplicate rows, so nothing was removed here.
3. Outliers (IQR)	Applied 1.5×IQR to five target-related features, only to non-bankrupt rows.	621 healthy rows were removed and all 220 bankrupt rows were kept. 6,819 → 6,198.
4. Scaling	Used MinMaxScaler on 94 features.	Useful for SVM and KNN; tree models are less affected by scaling.
5. Correlation selection	Dropped 32 features with $\|r\| < 0.02$ with the target.	Removed features with very little direct relationship with the target. 94 → 62.
6. Feature engineering	Created Profitability Ratio = ROA(C) / (ROA(A) + 0.0001).	Added a feature combining two profitability measures. 62 → 63.
7. Stratified split	80/20 split, random state = 42, stratify = y.	Kept the class proportion similar: 4,958 train and 1,240 tests.

The final data had 176 bankrupt companies in training and 44 in testing. Since bankrupt companies were only about 1 in 28, the models could easily favour the healthy class.

3.1 Preprocessing Steps in Detail

The figures in this section show the effect of the main preprocessing steps. The step numbers match the table above.

Steps 1 and 2: constant column, missing values and duplicates. Figure 3 shows how the number of rows and features changed. Removing Net Income Flag reduced the 95 features to 94. The missing values and duplicates check made no changes, since the dataset had none. IQR outlier removal reduced the rows from 6,819 to 6,198, and correlation-based selection reduced the features from 94 to 62. The new ratio brought the number of features to 63.

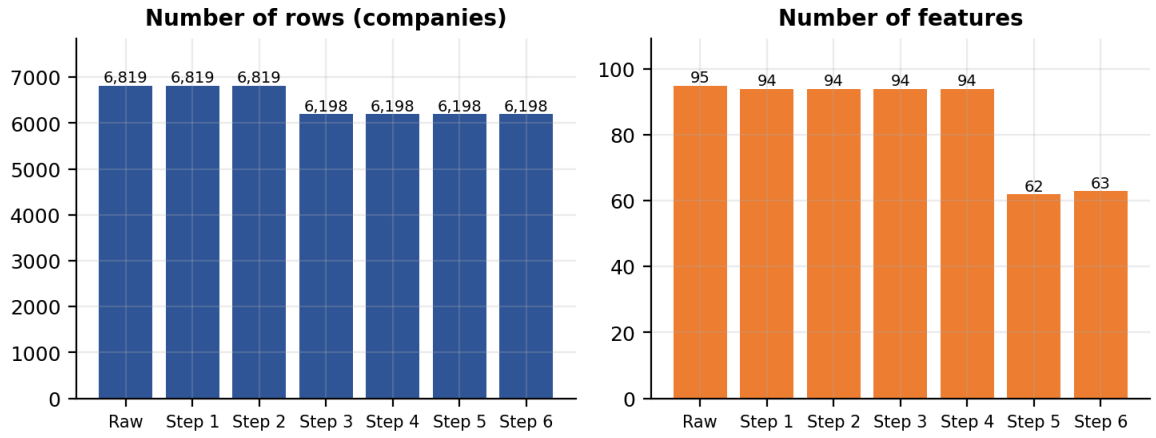


Figure 3. Number of rows (left) and number of features (right) after each preprocessing step.

Step 3: IQR outlier removal. Figure 4 shows the five target-related features used for the 1.5 × IQR rule. Before the rule, healthy companies had many extreme values; after it the ranges are much tighter. The rule was applied only to non-bankrupt rows, so 621 healthy companies were removed and all 220 bankrupt companies were kept. This is important since bankrupt companies are rare and may have unusual ratios and would lose the signal that the models need.

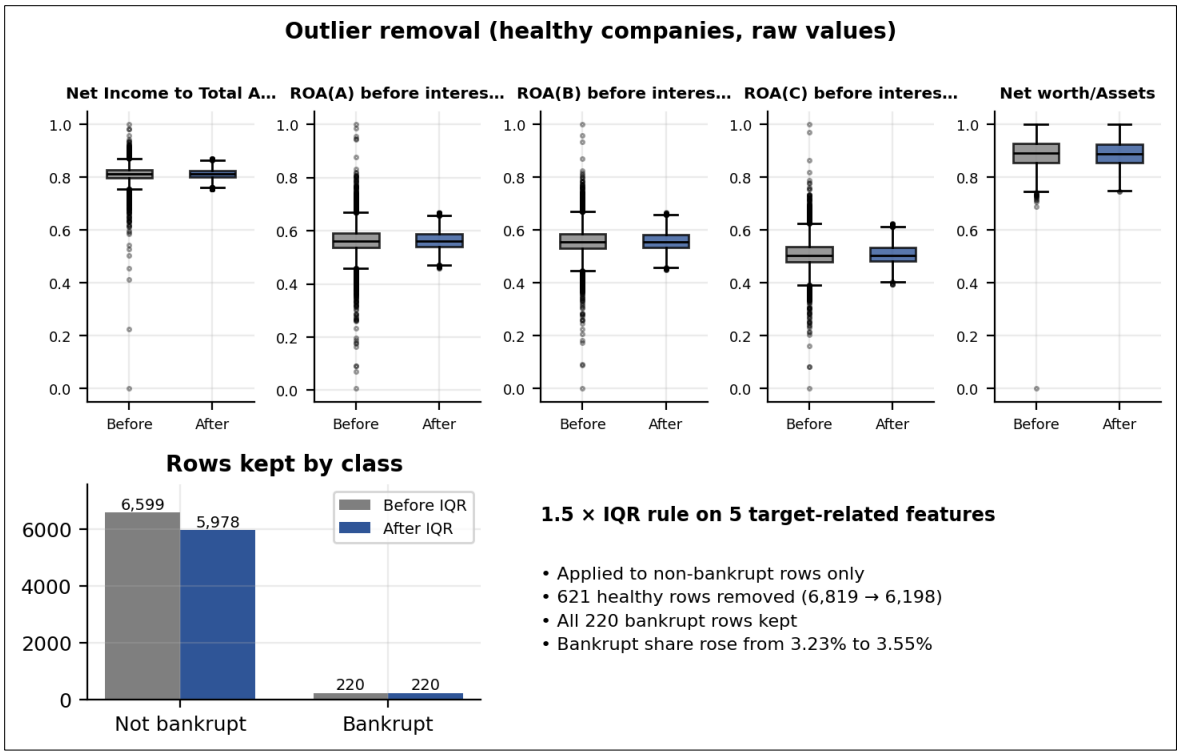


Figure 4. Boxplots of the five IQR features for healthy companies before and after outlier removal (left and top), and the number of rows kept by class.

Step 4: MinMax scaling. MinMaxScaler converts each feature to the range 0 to 1 using (x − min) / (max − min). Figure 5 shows that the shape of each distribution remains the same and only the range changes. For example, Cash Turnover Rate reached about 1e10 before scaling, while Persistent EPS stayed below 0.5. Scaling is important for SVM and KNN, which use distances, and matters little for tree models. One weakness is that a feature with a few very large values, such as Quick Ratio (maximum about 9.2e9), is squeezed close to zero for most companies after scaling.

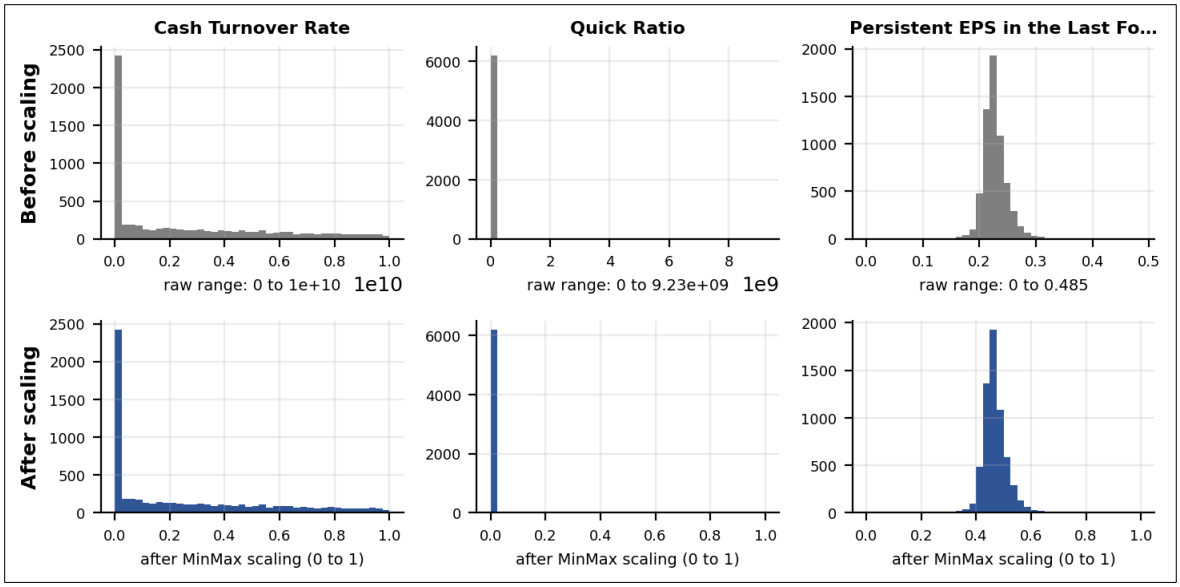


Figure 5. Distribution of three features before (top) and after (bottom) MinMax scaling.

Step 5: correlation-based feature selection. Figure 6 (left) shows the absolute correlation of all 94 features with the target. The 32 features below the cut-off of 0.02 were dropped and the 62 features above it were kept. The strongest features are Net Income to Total Assets (|r| = 0.47), ROA(A) (0.42), ROA(B) (0.39), Retained Earnings to Total Assets (0.38) and ROA(C) (0.37); all of them are negatively correlated with bankruptcy, which means that companies with higher profitability are less likely to be bankrupt. Correlation measures only linear relationships, so a feature with a low |r| could still help a non-linear model. The cut-off of 0.02 removes only features with almost no linear link to the target.

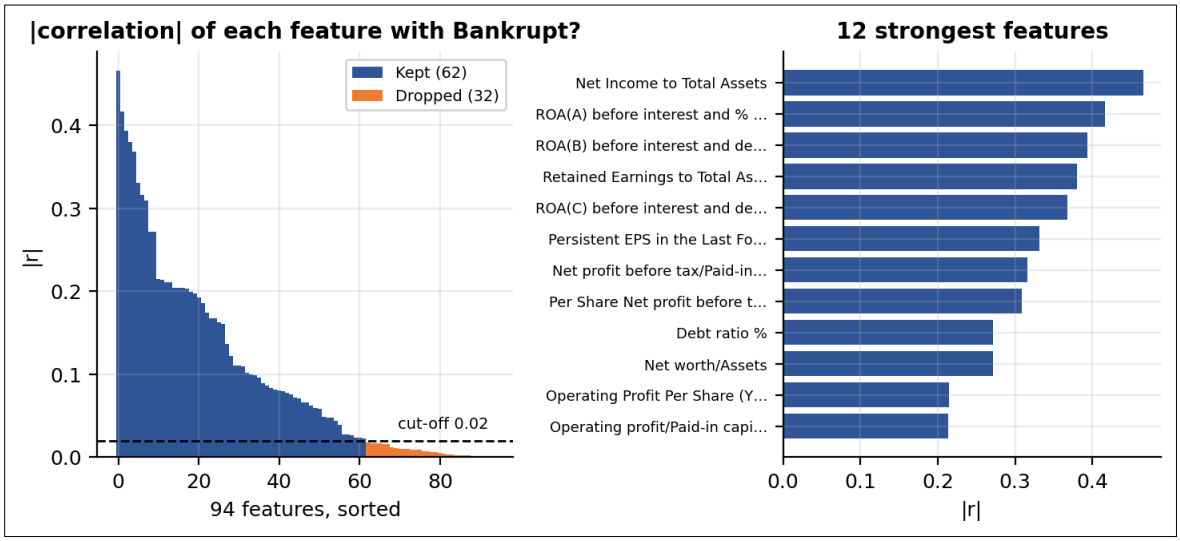


Figure 6. Absolute correlation of each feature with the target, with the 0.02 cut-off (left), and the 12 strongest features (right).

Step 6: feature engineering. The new feature Profitability Ratio = ROA(C) / (ROA(A) + 0.0001) compares two measurement of profitability. Figure 7 shows that the median is similar for both classes (0.953 for healthy and 0.938 for bankrupt companies), but the spread is wider for bankrupt companies (IQR 0.051 against 0.037), and a few bankrupt companies have extreme values. The scatter plot shows that the two ROA measures are strongly related, so the ratio is close to 1 for most companies. The ratio was calculated after scaling, which is why its maximum value is above 1 (5.12).

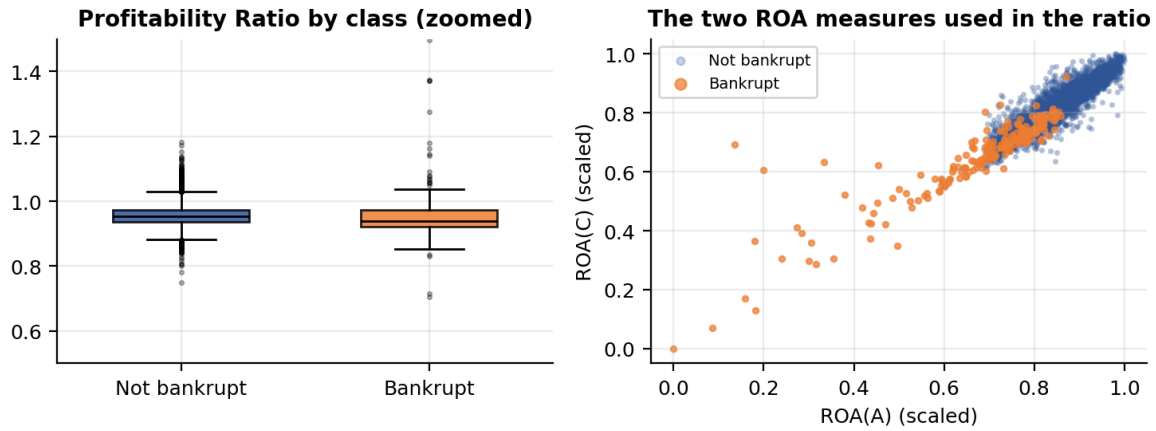


Figure 7. Profitability Ratio for healthy and bankrupt companies (left, zoomed) and the two scaled ROA measures used to build it (right).

Step 7: stratified split. The split into 4,958 training and 1,240 test companies is shown in Figure 2. The share of bankrupt companies is 3.55% in both sets. The scaler and the correlation filter were fit on the whole dataset before the split, which is a limitation explained in Section 5.7.

3.2 Exploratory Data Analysis

The exploratory analysis also revealed strong correlations between many features. For example, 23 feature pairs had |r| > 0.9. This is important for Naive Bayes since it assumes that features are independent.

Figure 8 (left) shows the correlation between the 12 features most closely related to bankruptcy and the target. These features are also strongly related to each other: the three ROA measures are highly correlated (for example ROA(A) and ROA(B), r = 0.94), and Debt ratio % and Net worth/Assets have r = -1.0, which means that they carry almost the same information. The histogram (right) shows that most of the 1,953 feature pairs have a weak correlation, but 23 pairs have |r| > 0.9. Tree-based models and SVM can work with correlated features, but Naive Bayes assumes independence, so it suffers the most (Section 4.8).

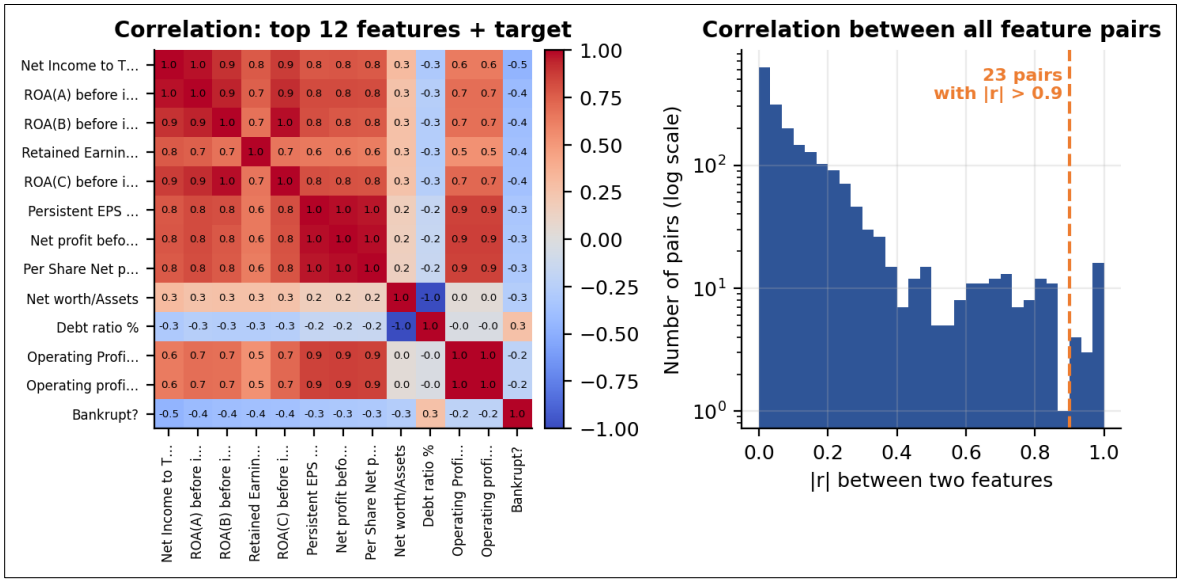


Figure 8. Correlation of the 12 strongest features with the target (left) and distribution of the correlation between all feature pairs (right).

4. Model Design and Implementation

4.1 Common Experimental Setup

All six models were implemented using the same 63-feature train/test files and the same random state. Each model was used in default, imbalance-aware and tuned forms.

- GridSearchCV with 3-fold stratified cross-validation was used for tuning.
- Average precision (PR-AUC) was used as the main tuning score since the bankrupt class is rare.
- Five-fold stratified cross-validation was used to check stability.
- The test set was used once for final evaluation.
- ROC curves, precision-recall curves and train/test comparisons were also examined.

4.2 The Six Models

Model	**Main idea**	**Imbalance handling / tuned setting**
SVM	Finds a decision boundary with a large margin; RBF allows a non-linear boundary.	Balanced class weights; C = 1, gamma = 0.05.
XGBoost	Builds trees one after another and corrects previous errors.	scale_pos_weight ≈ 27; 300 trees, depth 2, learning rate 0.03.
KNN	Predicts using nearby companies based on distance.	No class-weight option; k = 25.
Gradient Boosting	Builds small trees sequentially, learning from remaining errors.	Balanced sample weights; 100 trees, depth 3, learning rate 0.05.
AdaBoost	Builds weak learners and focuses more on misclassified examples.	Balanced starting weights; 200 stumps, learning rate 0.1.
Gaussian Naive Bayes	Models each feature for each class and assumes feature independence.	Equal priors; var_smoothing = 1e-3.

SVM needs scaled inputs and benefits from class weights since the bankrupt class is small. XGBoost uses boosted trees and has a built-in imbalance parameter. KNN is affected by the large number of healthy companies since nearby companies are mostly healthy. Gradient Boosting also builds trees sequentially, and balanced sample weights were used since the scikit-learn implementation does not provide class weight. AdaBoost increases the importance of difficult examples, which can improve recall but also create false alarms. Gaussian Naive Bayes is fast, but its independence assumption does not hold in this dataset with strong correlations.

4.3 SVM (RBF Support Vector Machine)

Responsible member: Parindya K. P. (IT25102104)

Suitability. The Support Vector Machine finds the boundary that separates the two classes with the widest margin. With the RBF kernel the boundary can be non-linear, suitable for financial ratios that interact in complex ways. SVM works best on scaled numeric inputs, and the data were already MinMax scaled. The training set (4,958 rows and 63 features) is small enough for an SVM to train quickly.

Implementation and tuning. SVC(kernel="rbf") was used with class_weight="balanced", so that an error on a bankrupt company costs about 27 times more than an error on a healthy company, and with probability=True, so that ROC-AUC and PR-AUC can be calculated. GridSearchCV (3-fold stratified cross-validation, scored by PR-AUC) searched C = 0.5, 1, 5, 10 and gamma = "scale", 0.01, 0.05, which gives 12 combinations. The selected values are C = 1 and gamma = 0.05.

Parameter	Purpose	Selected value
C	Penalty for misclassified training companies. A larger C gives a narrower margin and a more complex boundary.	1
gamma	How far the influence of one training company reaches. Small values give a smoother boundary.	0.05
class_weight	Makes errors on the rare bankrupt class more costly.	balanced
kernel	Shape of the decision boundary.	RBF

Imbalance handling and variants. Without any imbalance handling the SVM detected only 18.2% of the bankrupt companies (recall 0.182). Balanced class weights raised the recall to 0.886, while PR-AUC changed from 0.713 to 0.653. SMOTE (recall 0.818, PR-AUC 0.608) and random under-sampling (recall 0.909, PR-AUC 0.626) gave a similar recall but a lower PR-AUC, so class weights were kept.

Setting	Recall	PR-AUC
No change	.182	.713
Class weights	.886	.653
SMOTE	.818	.608
Under-sampling	.909	.626

Test-set result. The selected model was evaluated once on the test set (1,240 companies, 44 bankrupt).

Selected model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Caught / false alarms
SVM (C = 1, gamma = 0.05, balanced)	.892	.232	.886	.368	.964	.665	39 / 129

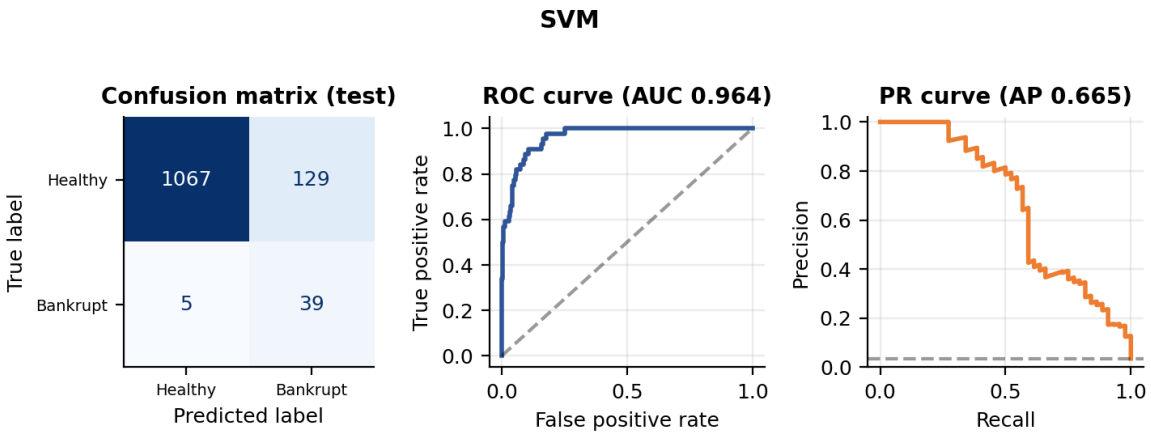


Figure 9. SVM test-set results: confusion matrix, ROC curve and precision-recall curve.

Figure 9 shows that the SVM detected 39 of the 44 bankrupt companies and missed 5, but it also marked 129 healthy companies as bankrupt. The ROC curve rises steeply (AUC 0.964), but the precision-recall curve falls rapidly as the recall increases (PR-AUC 0.665). With so few bankrupt companies, a high number of false alarms massively lowers the precision.

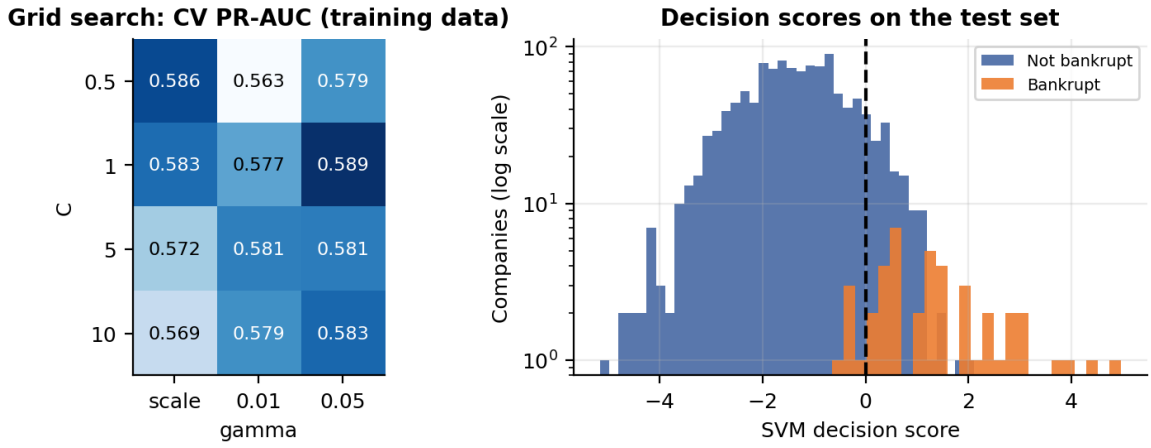


Figure 10. SVM grid-search scores (left) and decision scores on the test set (right).

Figure 10 (left) shows that the grid-search scores are close together (0.563 to 0.589), so the SVM is not highly sensitive to C and gamma in this range; the best cell is C = 1 and gamma = 0.05. On the right, bankrupt companies mostly have higher decision scores, but their range overlaps with the healthy companies near the decision boundary (score 0), causing the false alarms.

Evaluation and interpretation. Accuracy was 0.892, precision 0.232, recall 0.886, F1 0.368, ROC-AUC 0.964 and PR-AUC 0.665. The SVM is a high-recall model: it finds most bankruptcies but with many false alarms. Its training F1 (0.350) and test F1 (0.368) are both low and close to each other, so it may suffer from underfitting rather than overfitting.

Limitations and possible improvements. The grid only contained 12 combinations. The probability output of SVC comes from an extra internal calibration step, so using 0.5 on predict_proba does not reproduce the same labels as predict(). SVM training time also increases rapidly as you add rows.

4.4 XGBoost

Responsible member: Sandeepana A. G. P. (IT25102100)

Suitability. XGBoost builds decision trees one after another, and each new tree tries to fix the errors of the precious trees. It can model non-linear relationships and interactions between ratios, it does not need scaled inputs, and it is not much affected by correlated features. It also has a built-in parameter for imbalanced data and regularisation to reduce overfitting.

Implementation and tuning. XGBClassifier was used with scale_pos_weight = 4,782 / 176 ≈ 27.2 (healthy companies divided by bankrupt companies in the training set). GridSearchCV (3-fold, PR-AUC) searched n_estimators = 100, 300; max_depth = 2, 4, 6; and learning_rate = 0.03, 0.1, which gives 12 combinations. The selected model uses 300 trees of depth 2 with a learning rate of 0.03.

Parameter	Purpose	Selected value
n_estimators	Number of boosting trees.	300
max_depth	Depth of each tree. Shallow trees reduce overfitting.	2
learning_rate	Contribution of each new tree. Small values learn slowly but usually generalise better.	0.03
scale_pos_weight	Weight of the bankrupt class compared with the healthy class.	≈ 27.2

Imbalance handling and variants. With the default settings XGBoost detected 54.5% of the bankrupt companies (recall 0.545) and had the best PR-AUC of all variants (0.785). Class weighting raised the recall to 0.659 (PR-AUC 0.752), SMOTE to 0.636 (0.689) and under-sampling to 0.909 (0.494). Under-sampling gives the highest recall, but it removes many healthy companies and lowers PR-AUC strongly.

Setting	Recall	PR-AUC
No change	.545	.785
Class weights	.659	.752
SMOTE	.636	.689
Under-sampling	.909	.494

Test-set result. The selected model was evaluated once on the test set (1,240 companies, 44 bankrupt).

Selected model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Caught / false alarms
XGBoost (300 trees, depth 2, learning rate 0.03)	.926	.303	.841	.446	.968	.791	37 / 85

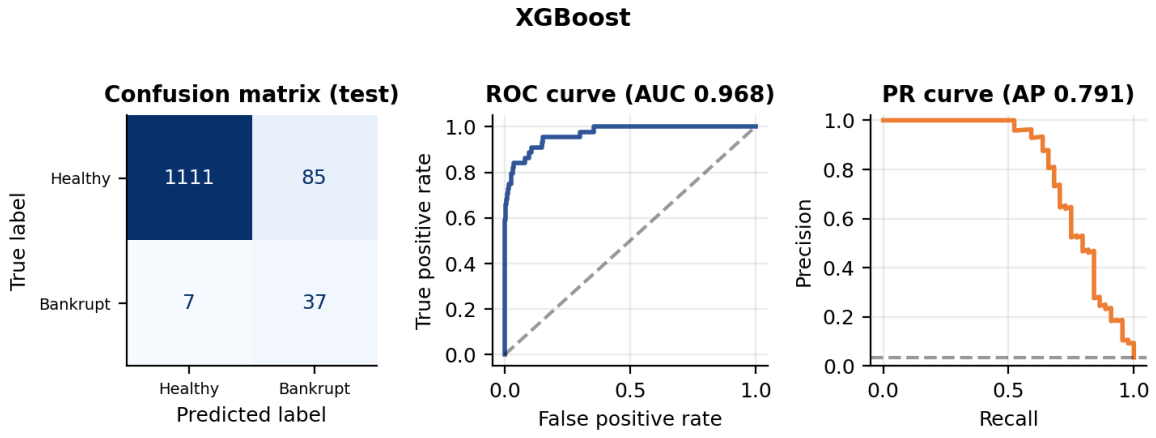


Figure 11. XGBoost test-set results: confusion matrix, ROC curve and precision-recall curve.

Figure 11 indicates that XGBoost classified 37 of 44 bankrupt companies correctly, which resulted in 85 false alarms. It also has the best ROC-AUC (0.968) and PR-AUC (0.791) among the six models.

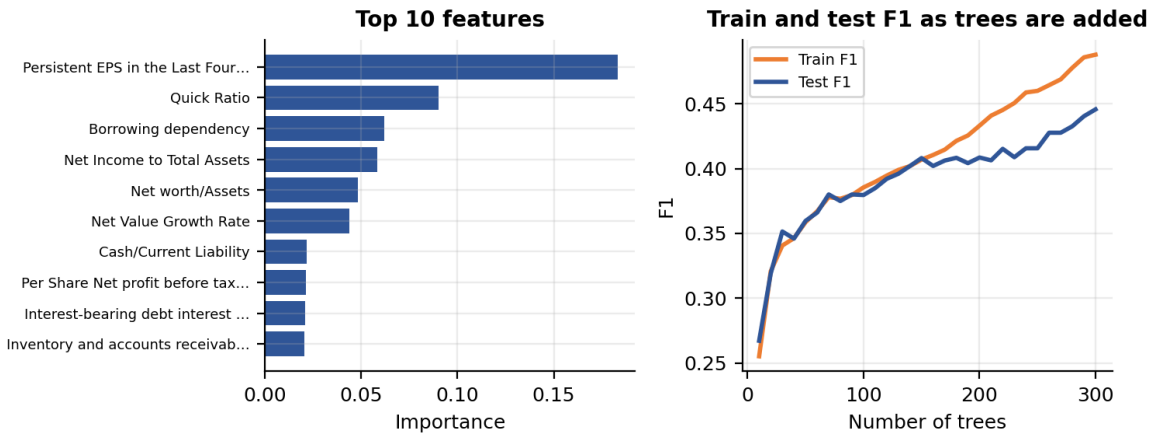


Figure 12. XGBoost feature importance (left) and train and test F1 as trees are added (right).

Figure 12 (left) shows that the most important feature is Persistent EPS in the Last Four Seasons (0.18), followed by Quick Ratio (0.09), Borrowing dependency (0.06), Net Income to Total Assets (0.06) and Net worth/Assets (0.05). On the right, the train and test F1 increase together up to around 100 trees (0.386 and 0.380). Beyond that the training F1 increases faster, and at 300 trees there is a small gap between them (0.488 and 0.446). This test-set curve is only a diagnostic and was not used to select the settings.

Evaluation and interpretation. Accuracy was 0.926, precision 0.303, recall 0.841, F1 0.446, ROC-AUC 0.968 and PR-AUC 0.791. Tuning for PR-AUC increased the model to a point where it had higher recall than the default model (0.841 against 0.545), at the expense of lower precision. The default model (depth 4, 200 trees) reached a training F1 of 1.000 against a test F1 of 0.706, a large gap indicative of overfitting; the tuned shallow trees narrowed this gap**.**

Limitations and possible improvements. Firstly, the grid only included 12 combinations, and the threshold was fixed at 0.5. Secondly, the feature importance only indicates which ratios the model used, not which ones actually predicted bankruptcy. Finally, since the test set only included 44 bankrupt companies, the estimate of the PR-AUC is not accurate.

.

4.5 KNN (K-Nearest Neighbours)

Responsible member: Raveenath K. A. Y. (IT25102101)

Suitability. K-Nearest Neighbours predicts the class of a company by taking votes from the k most similar companies, where similarity is defined as the distance between the scaled ratios. It is simple, makes no assumptions about the shape of the data and is easy to explain. Its use of distances requires scaled features, which were provided by the preprocessing

Implementation and tuning. KNeighborsClassifier was used with the distance weighting, so closer neighbours get more votes, was used. GridSearchCV (3-fold, PR-AUC) searched k = 3, 5, 9, 15 and 25. The selected value is k = 25: the cross-validated PR-AUC goes from 0.312 at k = 3 to 0.502 at k = 25.

Parameter	Purpose	Selected value
n_neighbors (k)	Number of nearby companies which vote for the class.	25
weights	"distance" gives closer companies a larger vote.	distance
metric	How the distance between companies is measured.	Euclidean (default)

Imbalance handling and variants. KNN has no class weight option, therefore the second row of the table represents distance weighting, rather than a balancing method. The recall was still 0.250 for both. SMOTE increased recall to 0.727 and under-sampling to 0.841, but PR-AUC decreased from 0.486 to 0.303 and 0.270, so the ranking quality became worse.

Setting	Recall	PR-AUC
No change	.250	.419
Distance weighting	.250	.486
SMOTE	.727	.303
Under-sampling	.841	.270

Test-set result. The selected model was evaluated once on the test set (1,240 companies, 44 bankrupt).

Selected model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Caught / false alarms
KNN (k = 25, distance weights)	.970	1.000	.159	.275	.865	.527	7 / 0

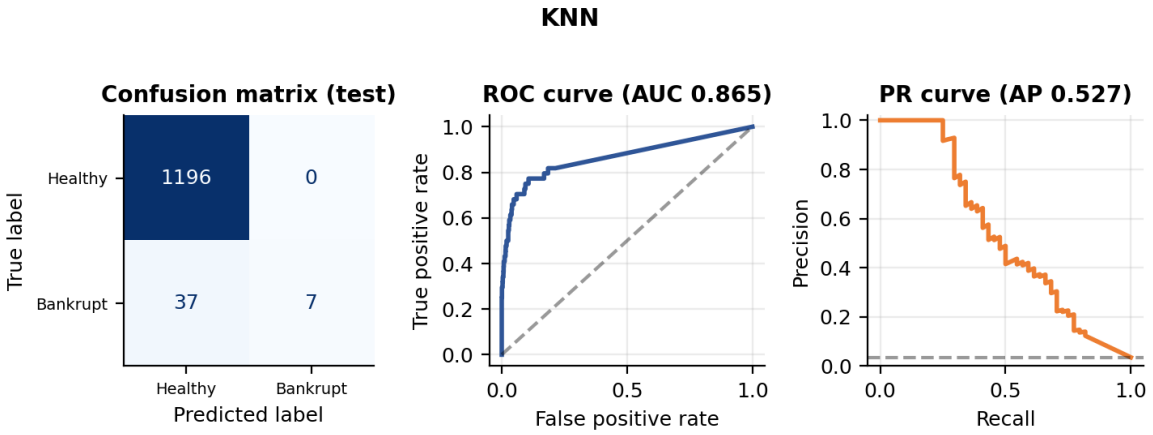


Figure 13. KNN test-set results: confusion matrix, ROC curve and precision-recall curve.

Figure 13 indicate that KNN only detected only 7 of the 44 bankrupt companies, but it had no false alarms (precision 1.000). The ROC-AUC (0.865) is the lowest of the six models, and the PR curve falls steeply.

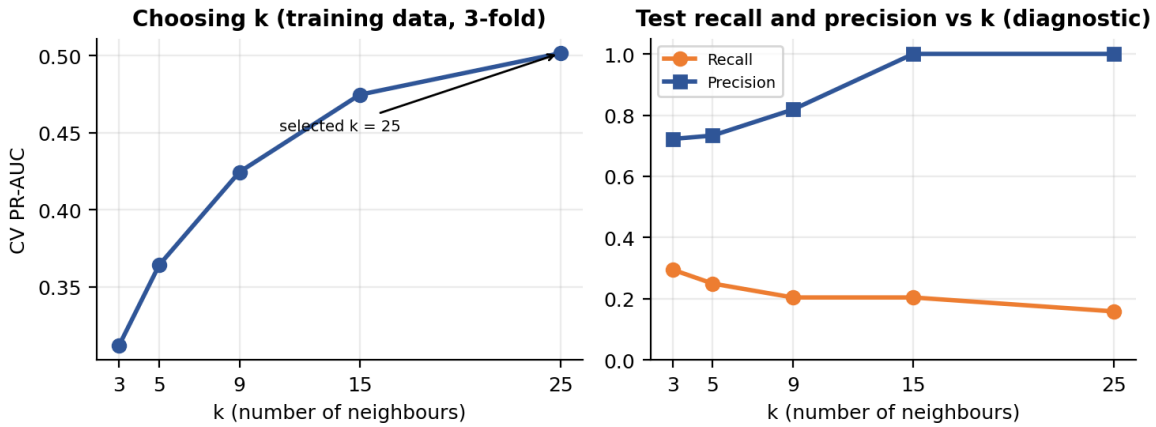


Figure 14. KNN: choosing k with cross-validation (left) and test recall and precision for different k (right).

Figure 14 (left) shows that with the tested values of k the cross-validated PR-AUC increases. The best value is at the edge of the grid (k = 25), so perhaps a larger k would be beneficial. On the right the test recall decreases from 0.295 at k = 3 to 0.159 at k = 25, while the precision increases from 0.722 to 1.000. With larger k the model becomes more conservative: it only predicts "bankrupt" when almost all of its neighbours are bankrupt.

Evaluation and interpretation. KNN had the highest accuracy of all the models (0.970) but the lowest recall (0.159): 37 out of the 44 bankrupt companies were missed. This is the clearest illustration in the project of why accuracy is a misleading metric on imbalanced data. The trained F1 after tuning is 1.000, but this is not real overfitting: with distance weighting each company is its own nearest neighbour (at distance zero).

Limitations and possible improvements. At 63 dimensions distances become less meaningful, and the healthy companies dominate most neighbourhoods. Correlated features are weighted multiple times in the distance, prediction is slow on large datasets, and the best k is at the edge of the grid.

4.6 Gradient Boosting

Responsible member: Rathnaweera R. P. S. D. (IT25102102)

Suitability. Gradient Boosting fits many small decision trees in succession, each correcting the remaining errors of the previous ones. It can capture non-linear patterns without any scaling and works well on tabular financial data.

Implementation and tuning. The scikit-learn GradientBoostingClassifier has no class weight option, and a small wrapper (BalancedGB) increases the sample weights of bankrupt companies in the fit(). Since the weights are applied inside fit(), the validation scores in cross-validation are not weighted. GridSearchCV (3-fold, PR-AUC) searched n_estimators = 100, 150; max depth = 2, 3 and learning rate = 0.05, 0.1, and there are 8 combinations. The selected model uses 100 trees, depth 3 and a learning rate of 0.05.

Parameter	Purpose	Selected value
n_estimators	Number of boosting trees.	100
max_depth	Depth of each tree.	3
learning_rate	Contribution of each new tree.	0.05
sample weights	Balanced weights, so bankrupt companies count about 27 times more.	balanced

Imbalance handling and variants. The default model detected 47.7% of the bankrupt companies (recall 0.477, PR-AUC 0.698). Balanced sample weights increased the recall to 0.750 and slightly increased PR-AUC to 0.709.

SMOTE (0.636 / 0.633) and under-sampling (0.886 / 0.507) decreased PR-AUC, so balanced weights were the best compromise.

Setting	Recall	PR-AUC
No change	.477	.698
Class weights	.750	.709
SMOTE	.636	.633
Under-sampling	.886	.507

Test-set result. The selected model was evaluated once on the test set (1,240 companies, 44 bankrupt).

Selected model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Caught / false alarms
Gradient Boosting (100 trees, depth 3, learning rate 0.05)	.931	.319	.818	.459	.958	.713	36 / 77

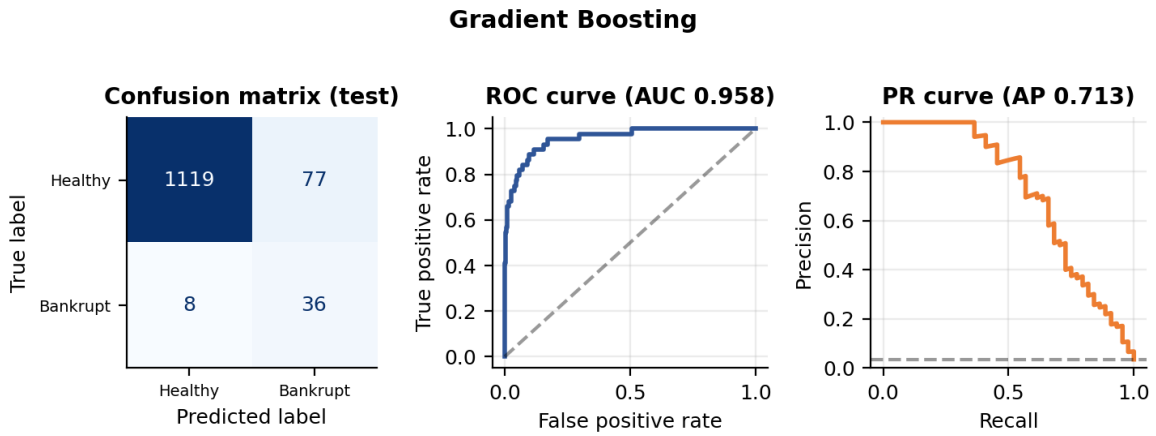


Figure 15. Gradient Boosting test-set results: confusion matrix, ROC curve and precision-recall curve.

Figure 15 shows that Gradient Boosting detected 36 of the 44 bankrupt companies with 77 false alarms, which is the fewest out of the four high-recall models. The ROC-AUC was 0.958 and the PR-AUC 0.713

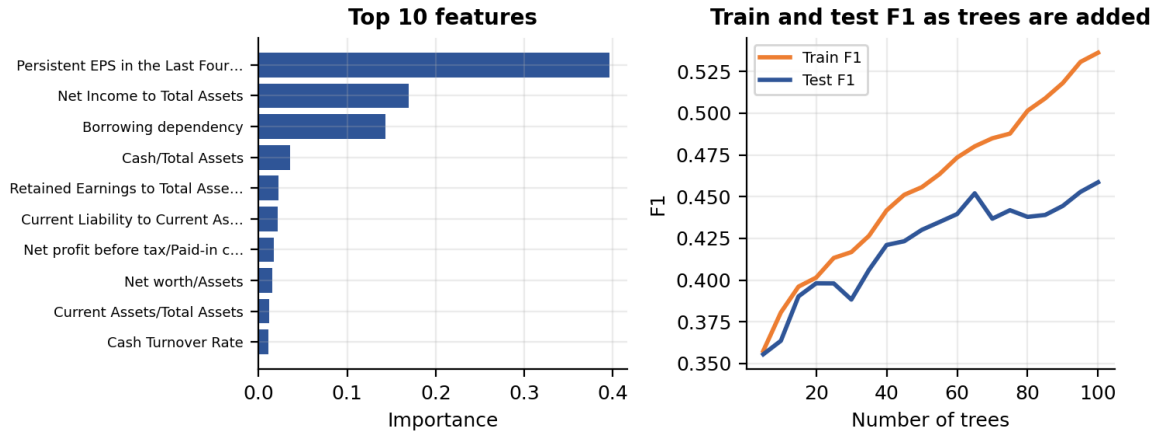


Figure 16. Gradient Boosting feature importance (left) and train and test F1 as trees are added (right).

Figure 16 (left) indicates that the model mainly uses three features: Persistent EPS in the Last Four Seasons (0.40), Net Income to Total Assets (0.17) and Borrowing dependency (0.14): these three together make up 71% of the importance. On the right the training F1 is consistently increasing with more trees (0.401 at 20 trees and 0.536 at

100), while the test F1 increases more slowly (0.398 and 0.459): a gap opens up as the model becomes more familiar with the training data.

Evaluation and interpretation. Accuracy was 0.931, precision 0.319, recall 0.818, F1 0.459, ROC-AUC 0.958 and PR-AUC 0.713. Gradient Boosting had the highest F1 of all the six tuned models and the second-highest PR-AUC. The default model overfit (training F1 0.920, test F1 0.592), but the gap was reduced to 0.536 against 0.459 by tuning.

Limitations and possible improvements. The grid only had 8 combinations, the threshold was left at 0.5 and the importance is concentrated in a few features. Like the other models, the results are based on only 44 bankrupt test companies.

4.7 AdaBoost

Responsible member: De Silva S. L. O. D. (IT25102105)

Suitability. AdaBoost combines many weak learners, in this case decision stumps (trees with a single split). After each round it increases the weight of companies classified incorrectly so that subsequent weak learners concentrate on the most problematic examples. This focus on difficult examples is important for rare classes.

Implementation and tuning. A wrapper (BalancedAda) gives balanced sample weights inside fit(), so that bankrupt companies have about 27 times the weight of healthy companies to start with. GridSearchCV (3-fold, PR-AUC) searched n_estimators = 50, 100, 200 and learning rate = 0.1, 0.5, 1.0, which are 9 combinations. The selected model uses 200 stumps and a learning rate of 0.1.

Parameter	Purpose	Selected value
n_estimators	Number of weak learners.	200
learning_rate	How strongly each weak learner changes the weights. Small values are more cautious.	0.1
base learner	Decision stump (depth 1), the default.	stump
sample weights	Balanced weights for the bankrupt class.	balanced

Imbalance handling and variants. The default model detected 38.6% of the bankrupt companies (recall 0.386, PR-AUC 0.684). Balanced weights increased the recall to 0.886 and PR-AUC changed slightly to 0.665. SMOTE (0.818 / 0.635) and under-sampling (0.886 / 0.641) gave similar results for recall and PR-AUC.

Setting	Recall	PR-AUC
No change	.386	.684
Class weights	.886	.665
SMOTE	.818	.635
Under-sampling	.886	.641

Test-set result. The selected model was evaluated once on the test set (1,240 companies, 44 bankrupt).

Selected model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Caught / false alarms
AdaBoost (200 stumps, learning rate 0.1)	.897	.244	.909	.385	.963	.699	40 / 124

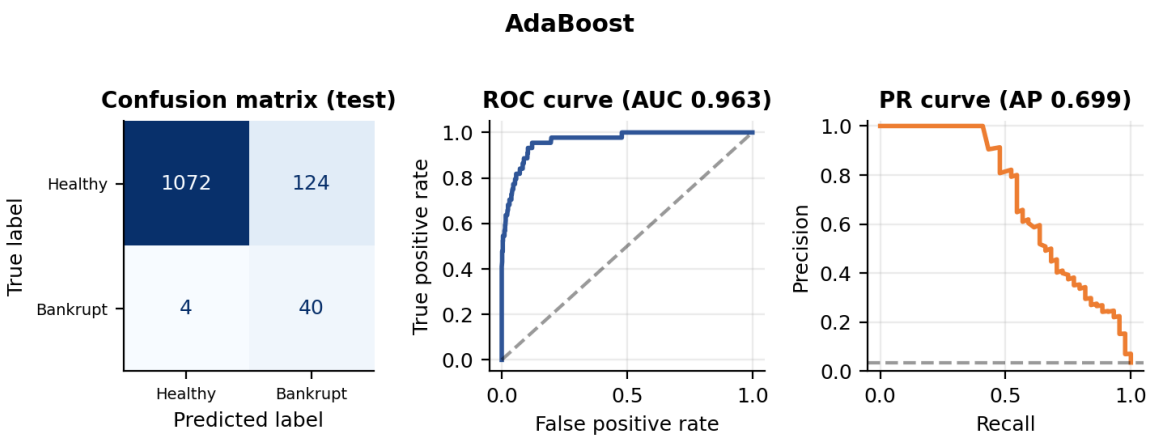


Figure 17. AdaBoost test-set results: confusion matrix, ROC curve and precision-recall curve.

Figure 17 shows that AdaBoost detected 40 of the 44 bankrupt companies, but it also generated 124 false alarms, the most of any boosting model. The ROC-AUC was 0.963 and PR-AUC 0.699.

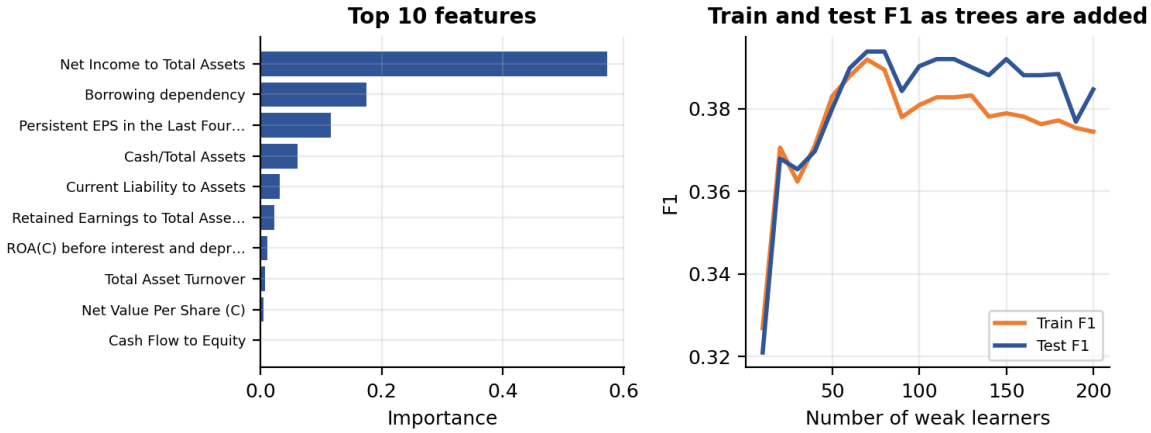


Figure 18. AdaBoost feature importance (left) and train and test F1 as weak learners are added (right).

Figure 18 (left) indicates that there is one dominant feature: Net Income to Total Assets has an importance of 0.57, and the top three features add up to 86%. This is expected because each stump uses only one feature. On the right the train and test F1 are both low and almost the same (about 0.37 to 0.39 from 20 learners onwards), so adding more weak learners makes little difference.

Evaluation and interpretation. Accuracy was 0.897, precision 0.244, recall 0.909, F1 0.385, ROC-AUC 0.963 and PR-AUC 0.699. AdaBoost is the most sensitive model for finding bankruptcies, but that comes at the cost of the most false alarms among boosting models. The training F1 (0.374) and test F1 (0.385) are close and low, so the stumps limit the model rather than overfitting.

Limitations and possible improvements. Stumps are very simple, so the model cannot capture complex interactions. AdaBoost also gives more weight to misclassified companies, so noisy or atypical companies can have too much influence. The grid only contained 9 combination.

4.8 Gaussian Naive Bayes

Responsible member: Adhikari A. A. P. P. (IT25102103)

Suitability. Gaussian Naive Bayes applies Bayes' rule with the assumption that features are independent within classes and follow a normal distribution, making the algorithm fast to train and having few hyperparameters to tune. However, in this dataset, the features are not independent as indicated by the strong correlations.

Implementation and tuning. The classifier was instantiated with priors = [0.5, 0.5] to make it slightly less biased towards the majority class. GridSearchCV (3-fold, PR-AUC) was used to select var_smoothing = 1e-3 from [1e-9, 1e-7, 1e-5, 1e-3] (4 options). Thus, the highest value was chosen to regularize the variances.

Parameter	Purpose	Selected value
priors	Starting probability of each class.	[0.5, 0.5]
var_smoothing	Small value added to every variance for stability. It also softens extreme probabilities.	1e-3

Imbalance handling and variants. Equal priors yielded the same result as the default (recall = 0.409, PR-AUC = 0.380), since the probabilities were almost always close to 0 or 1 (see Figure below). SMOTE improved recall and PR-AUC to 0.455 and 0.437, respectively; Naive Bayes is the only classifier for which SMOTE improved the PR-AUC. Under-sampling decreased the metrics.

Setting	Recall	PR-AUC
No change	.409	.380
Equal priors	.409	.380
SMOTE	.455	.437
Under-sampling	.386	.321

Test-set result. The final model was evaluated once on the test set (1,240 companies, 44 bankrupt).

Selected model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Caught / false alarms
Naive Bayes (var_smoothing = 1e-3)	.963	.469	.341	.395	.898	.400	15 / 17

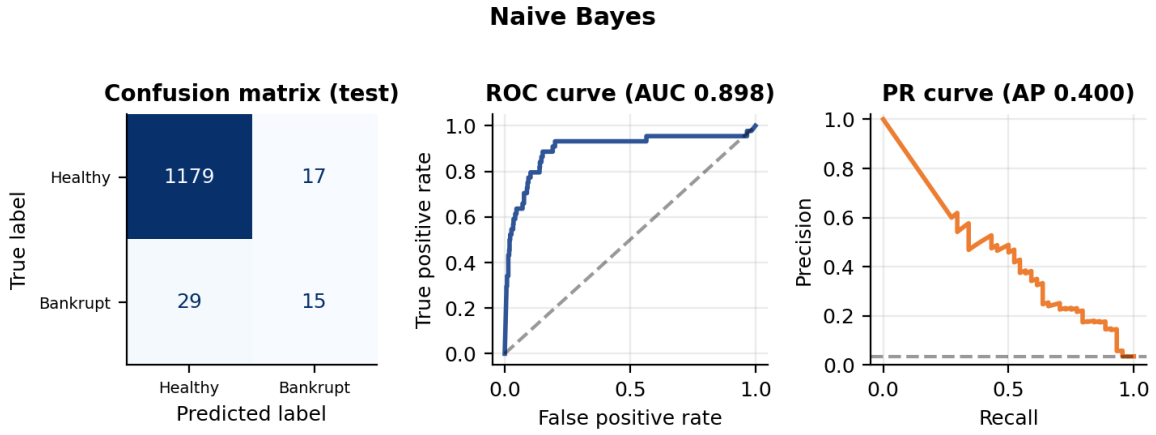


Figure 19. Naive Bayes test-set results: confusion matrix, ROC curve and precision-recall curve.

Figure 19 shows that Naive Bayes correctly identified 15 out of 44 bankrupt companies and produced 17 false alarms, thus achieving a recall of 0.341. It missed 29 bankrupt firms and had an ROC-AUC of 0.898 and PR-AUC of 0.400 – the lowest among the six approaches.

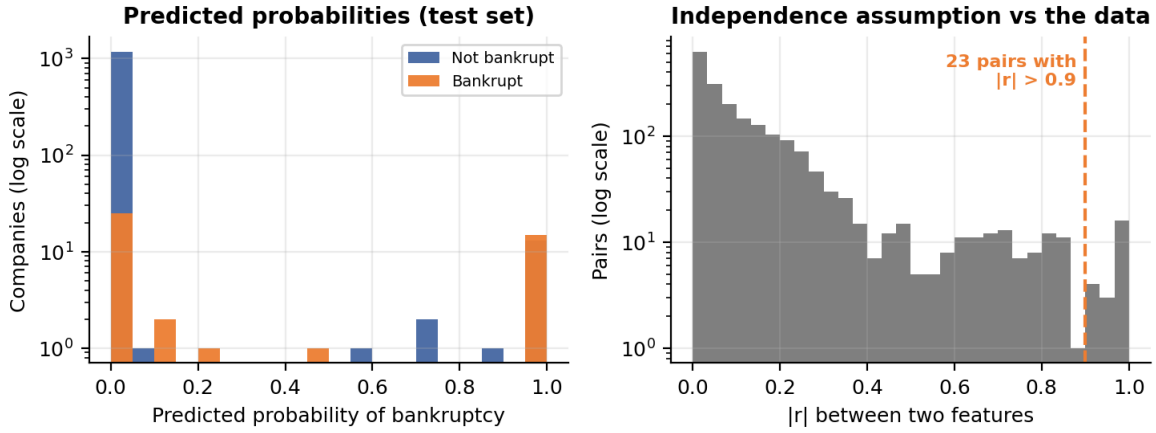


Figure 20. Naive Bayes predicted probabilities on the test set (left) and the correlation between feature pairs (right).

The left plot in Figure 20 demonstrates that 99.2% of the test-set predictions are below 0.01 or above 0.99, meaning that the model is extremely confident about its predictions and adjusting the decision threshold would have a limited impact. The right plot shows that there are 23 pairs of features with |r| > 0.9, which violates the independence assumption.

Evaluation and interpretation. Accuracy was 0.963, precision 0.469, recall 0.341, F1 0.395, ROC-AUC 0.898 and PR-AUC 0.400. Overall, Naive Bayes had a low number of false alarms but failed to identify nearly two-thirds of bankrupt companies. The F1 score on the training set was 0.527, which was not much higher than the test value of 0.395.

Limitations and possible improvements. The independence and normality assumptions were violated due to the correlated and skewed financial ratios. The best hyperparameter (var_smoothing) was at the edge of the search grid, and the predicted probabilities were poorly calibrated.

4.9 Individual Contributions

Each team member was responsible for one preprocessing technique and one model notebook. In this way, everyone could work individually while following the same scheme.

5. Evaluation and Comparison

5.1 Metrics

The metrics were computed for the bankrupt class as precision shows how many predicted bankrupt companies are actually bankrupt, while recall indicates how many bankrupt companies were predicted as bankrupt. F1 combines precision and recall into one metric. The area under the ROC curve (ROC-AUC) and precision-recall curve (PR-AUC) estimate the ranking quality: the higher the value, the better the model differentiates between the classes. For imbalanced datasets, the PR-AUC is more informative than ROC-AUC. Accuracy complements the metrics, as it demonstrates why the precision and recall are not informative for an imbalanced case.

5.2 Validation Method

The test set contained 1,240 companies, including 44 of which were bankrupt, and it was used to report the final performance of the models. The hyperparameters were tuned using a 3-fold stratified grid search on the training data. A 5-fold stratified cross-validation was used to estimate the stability of the results. SMOTE and random under-sampling were applied only within the training folds when used.

5.3 Main Results – Tuned Models

Model	**Accuracy**	**Precision**	**Recall**	**F1**	**ROC-AUC**	**PR-AUC**	**Caught / false alarms**
SVM	.892	.232	.886	.368	.964	.665	39 / 129
XGBoost	.926	.303	.841	.446	.968	.791	37 / 85
KNN	.970	1.000	.159	.275	.865	.527	7 / 0
Gradient Boosting	.931	.319	.818	.459	.958	.713	36 / 77
AdaBoost	.897	.244	.909	.385	.963	.699	40 / 124
Naive Bayes	.963	.469	.341	.395	.898	.400	15 / 17

The results demonstrated that multiple metrics should be considered to select the best model. For example, KNN achieved the highest accuracy but identified only 7 out of 44 bankrupt companies, while AdaBoost had the best recall but produced the most false alarms. XGBoost had the best PR-AUC and ROC-AUC, and Gradient Boosting had the highest F1 among the six classifiers.

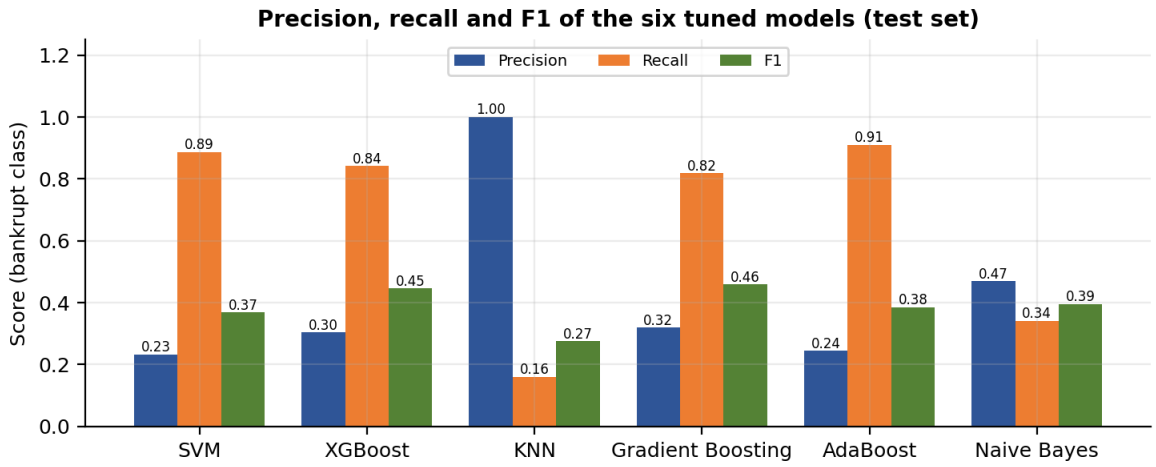


Figure 21. Precision, recall and F1 of the six tuned models on the test set.

Figure 21 shows the trade-off between precision and recall: KNN had perfect precision but almost zero recall, and AdaBoost and SVM had the highest recall but low precision. The best balance was achieved by XGBoost and Gradient Boosting with the F1 score of 0.446 and 0.459, respectively.

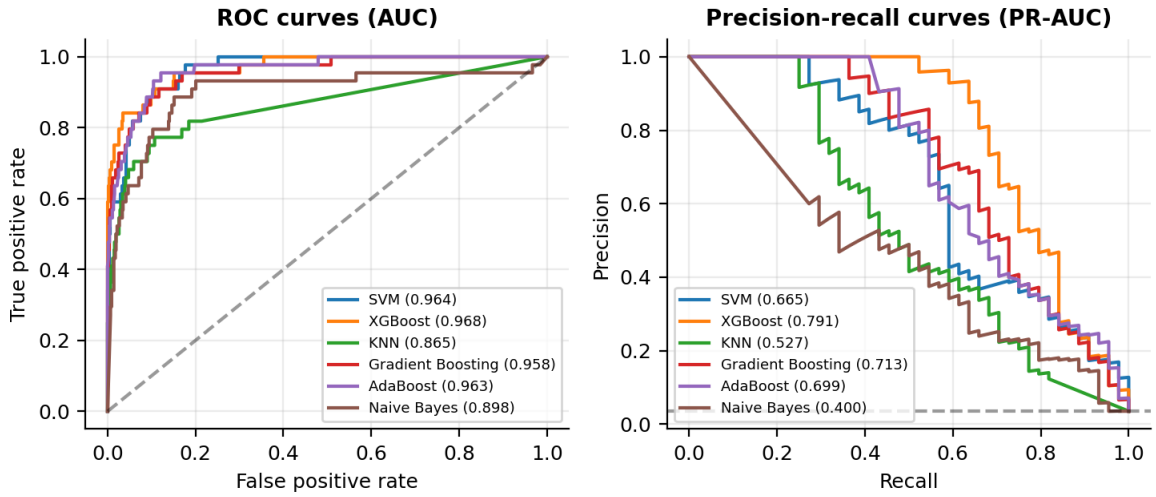


Figure 22. ROC curves (left) and precision-recall curves (right) of the six tuned models on the test set.

The Figure above compares the ranking quality of the six classifiers. Four of them (SVM, XGBoost, Gradient Boosting, AdaBoost) had a high ROC-AUC between 0.958 and 0.968, indicating that the AUC values were close to the maximum of 1. The precision-recall curves showed a bigger difference: XGBoost had the best PR-AUC of 0.791, followed by Gradient Boosting (0.713), AdaBoost (0.699), SVM (0.665), KNN (0.527) and Naive Bayes (0.400).

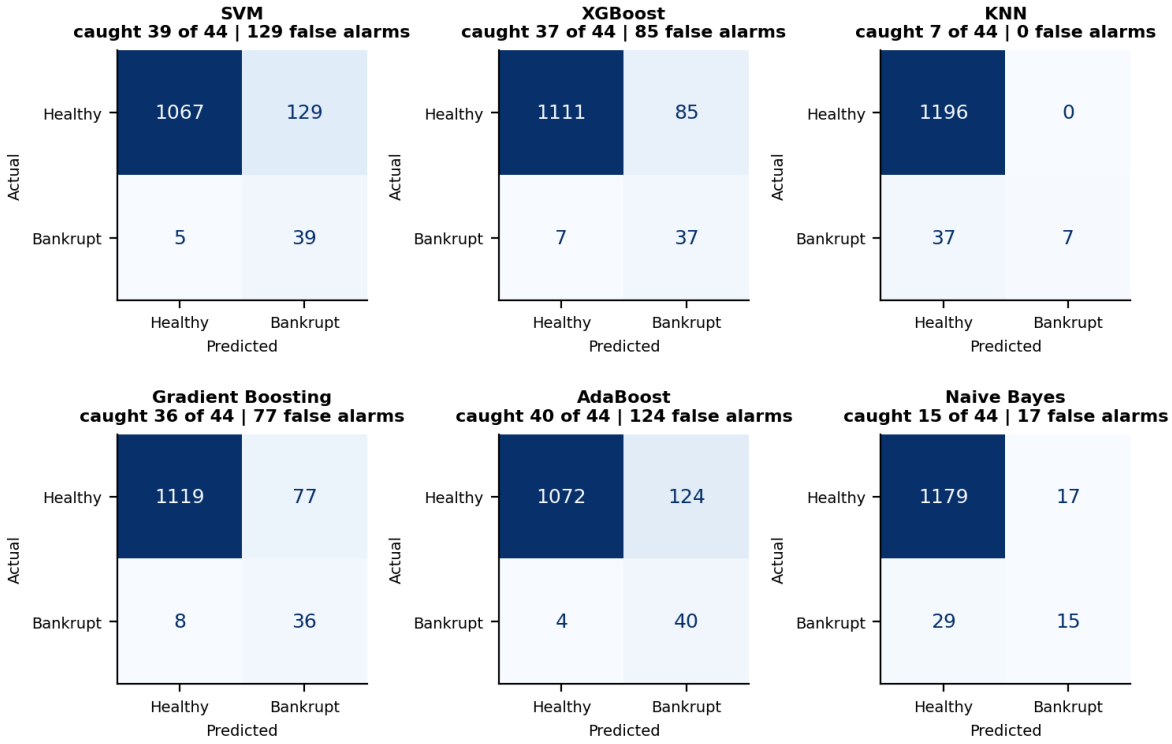


Figure 23. Confusion matrices of the six tuned models on the test set.

The Figure above shows the number of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). The best performing model (AdaBoost) had only 4 FN (missed bankrupt companies) and 124 FP (false alarms). The other models varied from 7 FN / 85 FP (KNN) to 37 FN / 77 FP (XGBoost). The number of false alarms greatly affected the precision: only KNN had 0 FN but also had 0 FP, hence, its precision was 1.

5.4 Effect of Imbalance Handling

Model	**No change (Recall/PR-AUC)**	**Class weights**	**SMOTE**	**Under-sampling**
SVM	.182 / .713	.886 / .653	.818 / .608	.909 / .626
XGBoost	.545 / .785	.659 / .752	.636 / .689	.909 / .494
KNN	.250 / .419	.250 / .486	.727 / .303	.841 / .270
Gradient Boosting	.477 / .698	.750 / .709	.636 / .633	.886 / .507
AdaBoost	.386 / .684	.886 / .665	.818 / .635	.886 / .641
Naive Bayes	.409 / .380	.409 / .380	.455 / .437	.386 / .321

The imbalance handling techniques generally improved recall but decreased precision, hence, the effect of these methods depended on the business context. Moreover, under-sampling decreased the PR-AUC for all models, as it removed potentially informative examples from the minority class, although it increased recall. The class weights, SMOTE, and under-sampling rarely improved the PR-AUC over the baseline.

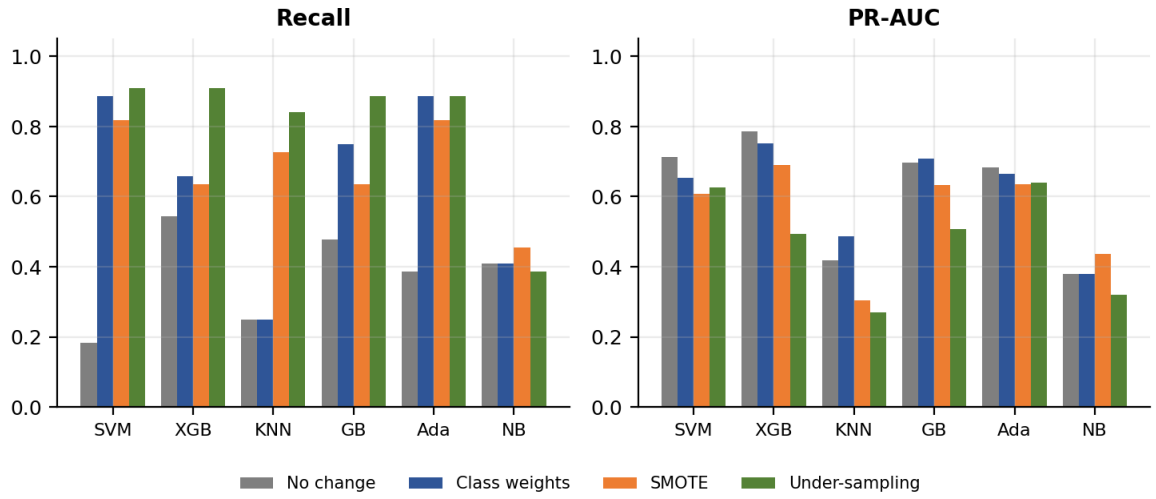


Figure 24. Recall (left) and PR-AUC (right) of each model with no change, class weights, SMOTE and under-sampling (values from the table above).

The Figure above demonstrates why the precision-recall metrics are more informative than accuracy for imbalanced classification tasks. Class weights improved the recall of SVM, AdaBoost, XGBoost, and Gradient Boosting to 0.886 – 0.750, but only Gradient Boosting had a better PR-AUC than the baseline. Under-sampling increased the recall of five out of six models to 0.909 – 0.841, but all had worse PR-AUC than the baseline.

5.5 Overfitting and Underfitting

Model	Default train/test F1	Tuned train/test F1	Observation
SVM	.249 / .308	.350 / .368	Underfits; both are low.
XGBoost	1.000 / .706	.488 / .446	Default overfits; tuning reduced the gap.
KNN	.500 / .373	1.000 / .275	Train score is affected by each row being its own neighbour.
Gradient Boosting	.920 / .592	.536 / .459	Default overfits; tuning reduced the gap.
AdaBoost	.600 / .540	.374 / .385	Weak learners give relatively low scores.
Naive Bayes	.473 / .468	.527 / .395	No large gap, but performance has a low ceiling.

For Gradient Boosting, the default model had a much higher training F1 of 0.920 than the test F1 of 0.592, which indicated overfitting. After tuning, the train/test F1 decreased to 0.536 / 0.459.

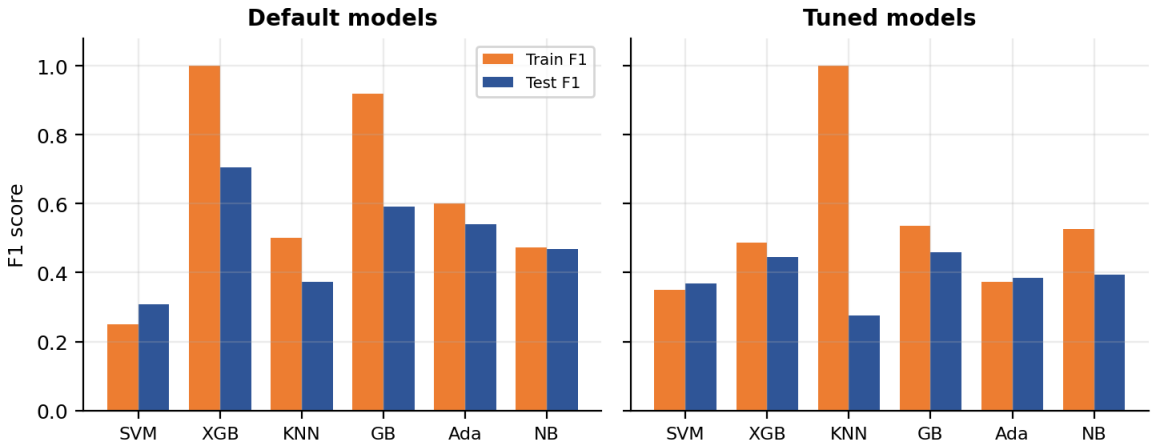


Figure 25. Train and test F1 of the default models (left) and the tuned models (right). Values are from the table above.

The Figure above demonstrates the overfitting / underfitting gap for all six classifiers. The default XGBoost, Gradient Boosting, and KNN models had a big train/test gap, which decreased after tuning for XGBoost and Gradient Boosting. SVM, AdaBoost, and Naive Bayes had a small train/test gap but achieved low scores, hence, they underfit.

5.6 Model Selection

XGBoost was found to be the best overall model since it had the highest test PR-AUC (0.791), test ROC-AUC (0.968) and cross-validated PR-AUC (0.683) among the six classifiers. It identified 37 bankrupt companies out of 44.

Gradient Boosting was another strong model with a test PR-AUC of 0.713 and F1 of 0.459. AdaBoost had the best recall of 0.909, while SVM also had a high recall. KNN had the best accuracy but the lowest recall among all classifiers, and Naive Bayes had the worst PR-AUC

The selection of the final model depended on the cost of false negatives and false positives: if one was significantly higher than the other, one should pick a model with a higher recall. Thus, it was advised to use the selected model as a support system rather than a fully automated decision-making system.

5.7 Limitations and Possible Improvements

- MinMaxScaler and correlation-based feature selection were fitted on the whole dataset instead of the training set, which introduced data leakage. It should be fixed by using a Pipeline.
- The test set only had 44 bankrupt companies, so the model performance could vary significantly for different test sets.
- The hyperparameter search space was small.
- The decision threshold should be selected using the validation set and business-specific costs rather than the test set.
- The dataset was limited to one market and one period, so the model may not generalize well to other populations.

Possible future improvements included nested cross-validation, a wider search space, threshold selection, other models such as Random Forest and Logistic Regression, and interpretability methods such as SHAP.

6. Ethical Considerations and Bias Mitigation

A false negative means a bankrupt company was predicted to be healthy, which could result in losses for the lenders or investors, while a false positive could damage the reputation of a healthy company. Hence, the model should not make final decisions but serve as an advisory support system.

The dataset was imbalanced and represented only one market and period, hence, the model might have biases against minority classes and companies that were not part of the training set.

The project used stratified sampling, cross-validation, compared different imbalance handling methods, did not discard any bankrupt companies when applying IQR outlier removal, and focused on the metrics for the minority class rather than accuracy.

7. AI Tool Usage Declaration and Transparency

AI tools were used as assistants during the project and helped to gain general knowledge, code review, model comparison, and improvements to the notebooks and documentation. The tools helped the team understand what methods and evaluation metrics to apply for the imbalanced classification task.

The final notebooks were executed on the group dataset and the reported results were double-checked with the results of the notebooks. The group is responsible for the final code, results, analysis, and conclusions, and each member can explain their contribution.

Tool	How it was used	How the output was checked
AI assistant	Guidance about algorithms, coding, model comparison, report structure and viva preparation.	Code was run on the project data and results were checked against notebook outputs.

8. Reflections and Lessons Learned

Technical Lessons

• Accuracy can be misleading for imbalanced datasets: for example, KNN achieved 0.970 accuracy but only 0.159 recall.

• Class weights and resampling techniques can improve recall but reduce precision, hence, it is important to choose the operating point based on business needs.

• The model with the best F1 at the default threshold was not necessarily the best in terms of ranking quality.

• A train-test gap should be evaluated along with the ranking performance to select the best model.

• Preprocessing has an impact on the results: using IQR outlier removal for the majority class only allowed to retain all 220 bankrupt companies.

Teamwork

Each group member worked individually on one preprocessing step and one model. It allowed to make the most of the time and compare the different approaches using the same train/test files and evaluation procedure. Whenever there were issues with class imbalance, model parameter settings, or evaluation results, the group would discuss them and review the code and outputs before finalizing the reports.

Challenges

The main challenges were to deal with class imbalance, tune the best performing models, and evaluate the results with appropriate metrics. Another challenge was to compare the different models which used different imbalance handling methods. Finally, the group discovered an error when using sample weights in GridSearchCV which artificially inflated the scores and fixed it before finalizing the reports.

Future Improvements

To improve the results, future work could focus on using more advanced techniques such as nested cross-validation, applying threshold selection, and implementing other models and interpretability methods such as SHAP.

9. References

Altman, E. I. (1968). Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. The Journal of Finance, 23(4), 589–609.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321–357.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.

Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273–297.

Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. Journal of Computer and System Sciences, 55(1), 119–139.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189–1232.

Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. Journal of Machine Learning Research, 18(17), 1–5.

Liang, D., & Tsai, C.-F. (n.d.). Taiwanese Bankruptcy Prediction [Data set]. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/572/taiwanese+bankruptcy+prediction

Liang, D., Lu, C.-C., Tsai, C.-F., & Shih, G.-A. (2016). Financial ratios and corporate governance indicators in bankruptcy prediction: A comprehensive study. European Journal of Operational Research, 252(2), 561–572.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLoS ONE, 10(3), e0118432.